

Hướng phát triển chuyên sâu cho dự án N-Puzzle Java 17 + Maven + JavaFX

Tóm tắt điều hành

Dự án hiện tại đã có một nền tảng tốt để chuyển từ **bài tập lớn môn Nhập môn AI** thành một **phòng thí nghiệm heuristic search** phục vụ đồng thời ba mục tiêu: giảng dạy, nghiên cứu thực nghiệm và benchmark. Báo cáo gốc của dự án đã bao gồm N-Puzzle kích thước 3×3, 4×4, 5×5; BFS, A*; nhiều heuristic từ misplaced tiles, Manhattan-like distance, Euclidean, sai hàng/cột đến linear conflict và heuristic H6; đồng thời đã có giao diện chơi, giải tự động và so sánh heuristic. [filecite0turn0file0](#)

Điểm quan trọng nhất trong giai đoạn tiếp theo **không phải bổ sung càng nhiều thuật toán càng tốt**, mà là biến codebase thành một nền tảng thí nghiệm có kiến trúc rõ ràng. Tôi khuyến nghị chuyển cấu trúc hiện tại từ các lớp kiểu `AStar`, `BFS`, `State`, `Node`, `N_PuzzleController` sang các mô-đun độc lập: `core`, `search-api`, `algorithms`, `heuristics`, `benchmark`, `ui`, rồi mới thêm `learning` và `distributed`. Java 17 đã có `ServiceLoader` để nạp service provider tại runtime, nên có thể xây dựng cơ chế plugin thuật toán/heuristic mà không cần một framework lớn. ¹

Về thuật toán, **IDA nên là ưu tiên cao nhất sau A**. Công trình kinh điển của Korf về iterative deepening cho thấy hướng tìm kiếm này đặc biệt phù hợp với cây tìm kiếm lớn khi bộ nhớ là giới hạn; Fifteen Puzzle chính là một trong các miền được sử dụng để chứng minh tính thực tiễn của cách tiếp cận. ² Sau IDA, nên triển khai RBFS để nghiên cứu trade-off thời gian-bộ nhớ, rồi SMA để cho phép người dùng đặt **memory budget** cụ thể. RBFS và IDA đều là các mô hình heuristic search không gian tuyến tính quan trọng trong nghiên cứu. ³ SMA đặc biệt có giá trị giảng dạy vì đặt giới hạn bộ nhớ tường minh, dù việc hiện thực đúng phức tạp hơn. ⁴

Về heuristic, bước nhảy lớn nhất sẽ đến từ **Pattern Database — PDB**, tiếp theo là **disjoint/additive PDB**. Pattern database lưu trước chi phí chính xác của các bài toán con và dùng chúng làm lower bound; trên 15-Puzzle, công trình Culberson–Schaeffer báo cáo mức giảm số node tìm kiếm hơn ba bậc độ lớn trên bộ benchmark được sử dụng trong nghiên cứu đó. ⁵ Korf–Felner sau đó cho thấy các pattern rời nhau có thể cộng chi phí mà vẫn giữ admissibility dưới điều kiện thích hợp, và báo cáo cải thiện rất lớn trên Fifteen Puzzle cũng như khả năng giải tối ưu các instance Twenty-Four Puzzle. ⁶ Additive PDB sau đó được tổng quát hóa sang cả partition tĩnh và động. ⁷

Một ưu tiên đặc biệt quan trọng là **kiểm định lại H5/H6 hiện có bằng dữ liệu exact**. Việc hai thành phần riêng lẻ đều là lower bound không đồng nghĩa tổng của chúng luôn là lower bound nếu cùng “tính” một phần chi phí. Đây chính là lý do nghiên cứu additive heuristics phải đưa ra điều kiện phân hoạch/disjoint cost một cách rõ ràng. ⁷ Vì vậy, thay vì chỉ chứng minh heuristic trên vài ví dụ như báo cáo hiện tại, phiên bản nghiên cứu nên duyệt toàn bộ không gian reachable của 3×3 bằng reverse BFS để tự động kiểm tra $h(s) \leq h^*(s)$ và consistency trên mọi cạnh.

MCTS nên xếp **ưu tiên thấp** cho solver N-Puzzle tối ưu. UCT/MCTS nguyên thủy được xây dựng cho Monte-Carlo planning trong không gian MDP lớn. ⁸ N-Puzzle cổ điển lại là deterministic shortest-path problem với action cost rõ ràng, nơi A/IDA/PDB tận dụng lower bounds trực tiếp hơn. MCTS chỉ đáng

thêm vào nếu mục tiêu là giảng dạy sự khác biệt giữa search paradigms, nghiên cứu policy learned, hoặc mở rộng bài toán sang môi trường stochastic/adversarial.

Về benchmark, cần tách **microbenchmark** khỏi **solver benchmark**. OpenJDK JMH khuyến nghị chạy benchmark trong một Maven project/subproject độc lập thay vì coi benchmark như JUnit test, vì khởi tạo JVM/JIT có thể làm sai lệch phép đo. ⁹ End-to-end experiments cần ghi lại seed, puzzle, goal, version code, Java/JVM, cấu hình heap, thuật toán, heuristic, preprocessing PDB, timeout và raw CSV/JSON. JDK Flight Recorder có thể cung cấp dữ liệu JVM/GC/diagnostic để phân tích các trường hợp chậm hoặc hết heap. ¹⁰

Nếu triển khai theo lộ trình khoảng **hai mươi bốn tuần**, giả định một người phát triển khoảng 15–20 giờ/tuần, sản phẩm cuối nên có:

1. Một **core solver library** chạy độc lập GUI.
2. A, IDA, RBFS, SMA* cùng một API.
3. Manhattan, Linear Conflict chuẩn hóa, PDB và additive PDB.
4. Benchmark suite tái tạo được.
5. JavaFX Search Visualizer + Replay + Compare Lab.
6. CI tự động kiểm thử correctness và heuristic properties.
7. Export dữ liệu đủ để tạo bảng/biểu đồ cho báo cáo khoa học.
8. Một nhánh nghiên cứu nâng cao cho learned heuristic và parallel/distributed search.

Ma trận ưu tiên tổng thể

Ưu tiên	Hạng mục	Mô tả ngắn	Lợi ích chính	Độ khó	Phụ thuộc kỹ thuật / ví dụ
P0	Search/Core API	Tách Board, Goal, SearchAlgorithm, Heuristic, SearchResult, SearchBudget khỏi JavaFX	Là nền cho mọi mở rộng sau này	Trung bình	Java 17, Maven multi-module
P0	Benchmark correctness	Reverse BFS 3×3, test optimality/admissibility/consistency	Ngăn kết luận sai về heuristic	Trung bình	JUnit parameterized tests ¹¹
P0	IDA*	Iterative deepening theo ngưỡng f=g+h	Giải puzzle lớn hơn với ít RAM	Trung bình	Core API, heuristic API; Korf ²
P0	State encoding hiệu năng cao	Packed state, goal lookup, precomputed moves	Giảm GC, hash cost, copy array	Trung bình	long cho ≤4×4, representation khác cho ≥5×5
P0	JavaFX async	Search chạy background; pause/cancel/progress	GUI không treo khi benchmark	Thấp-TB	Task / Service JavaFX ¹²

Ưu tiên	Hạng mục	Mô tả ngắn	Lợi ích chính	Độ khó	Phụ thuộc kỹ thuật / ví dụ
P0	CI + reproducibility	Build/test mỗi commit, lưu report/artifact	Biến dự án thành codebase nghiên cứu đáng tin	Thấp	Maven Surefire, JUnit, JaCoCo, GitHub Actions ¹³
P1	Pattern Database	Precompute abstract distances	Heuristic mạnh hơn Manhattan/LC	Cao	Reverse abstract search, PDB persistence ⁵
P1	Additive/disjoint PDB	Cộng PDB của các tập tile được cost-partition đúng	Mạnh hơn single PDB	Cao	Partition manager, admissibility proof ¹⁴
P1	RBFS	Best-first behavior trong không gian tuyến tính	Nghiên cứu time-memory trade-off	Trung bình	Search API; RBFS literature ³
P1	SMA*	A* với memory budget tường minh	Giá trị giảng dạy/bộ nhớ rất cao	Cao	Priority structures, backup <code>f</code> values ⁴
P1	Compare Lab	Chạy cùng input trên nhiều solver/heuristic song song	Trực quan hóa benchmark	Trung bình	Experiment runner, JavaFX charts ¹⁵
P1	Experiment export	CSV/JSON + manifest + charts	Dùng trực tiếp cho báo cáo/paper	Trung bình	Benchmark module
P2	Custom goal / arbitrary N	Goal không cố định; 3×3, 4×4, 5×5...	Nâng tính tổng quát	Trung bình	Generalized parity, Goal abstraction
P2	Parallel batch benchmark	Nhiều instance chạy đồng thời	Tăng throughput thí nghiệm	Trung bình	Dedicated <code>Executor</code> , <code>CompletableFuture</code> ¹⁶
P2	Learned heuristic	NN/RL học <code>h</code> hoặc policy	Hướng luận văn/paper hiện đại	Cao	Dataset, training pipeline, bounded-suboptimal search ¹⁷
P2	HDA*/distributed	Phân state theo hash giữa worker	Khai thác distributed RAM	Rất cao	Network/message layer, Zobrist/hash ownership ¹⁸

Ưu tiên	Hạng mục	Mô tả ngắn	Lợi ích chính	Độ khó	Phụ thuộc kỹ thuật / ví dụ
P3	MCTS/UCT	Sampling-based search baseline	Giá trị giảng dạy hơn là solver tối ưu	Cao	Simulation policy, budget model ⁸
P3	Adversarial generator	Sinh puzzle cổ tình “đánh lừa” heuristic	Nghiên cứu failure modes	Cao	Dataset generator, exact/reference solver

Kết luận ưu tiên: nếu mục tiêu gần nhất là một sản phẩm có thể dùng cho khóa luận hoặc bài báo thực nghiệm, thứ tự đầu tư nên là **Core API → correctness benchmark → IDA → PDB → Compare/Visualizer → RBFS/SMA → reproducible experiment pipeline**. Learned heuristic và distributed search chỉ nên bắt đầu khi pipeline benchmark cổ điển đã ổn định.

Thuật toán và heuristic

A* nên trở thành thuật toán chuẩn tham chiếu thay vì chỉ là một chức năng của GUI. Bài báo gốc của Hart, Nilsson và Raphael xây dựng cơ sở hình thức cho heuristic graph search và các tính chất tối ưu trong những điều kiện tương ứng. ¹⁹ Trong dự án mới, A* nên xuất ra đầy đủ `expanded`, `generated`, `duplicates`, `reopened`, `maxOpen`, `maxClosed`, `heuristicCalls`, `solutionCost`, `elapsedNs` thay vì chỉ time/node count. Đồng thời phải ghi rõ policy khi gặp một trạng thái với `g` tốt hơn: reopen hay không reopen; tie-break theo `f`, `h`, `g`, insertion order; những lựa chọn này phải nằm trong experiment configuration chứ không bị hard-code.

IDA* — ưu tiên P0. Thuật toán nên dùng ngưỡng `f=g+h`; mỗi iteration thực hiện depth-first search và ngưỡng mới là giá trị nhỏ nhất vượt ngưỡng hiện tại. Giá trị nghiên cứu của IDA* ở N-Puzzle rất rõ: Korf chỉ ra iterative deepening có thể đạt hiệu quả thời gian/không gian tốt trên các cây tăng trưởng theo hàm mũ và áp dụng trực tiếp vào Fifteen Puzzle. ²

Lợi ích lớn nhất của IDA là *không phải giữ toàn bộ OPEN/CLOSED như A*. Nhược điểm là regeneration: nhiều node có thể bị sinh lại qua các iteration. Đây chính là một thí nghiệm rất tốt cho dự án: cùng Manhattan/PDB, đo *A dùng bao nhiêu RAM* và IDA phải tái sinh bao nhiêu node. Độ khó: **trung bình**. Phụ thuộc: `Board` nhanh, `Heuristic`, cycle detection trên current path, cancellation token và search budget.

Nên triển khai hai phiên bản:

```
IDA*-basic
IDA*-TT      // có bounded transposition table
```

Từ đó có thể trả lời câu hỏi nghiên cứu: **thêm X MB transposition table giúp giảm node regeneration bao nhiêu?**

RBFS — ưu tiên P1. RBFS là lựa chọn tự nhiên tiếp theo vì đây cũng là heuristic search dùng không gian tuyến tính nhưng không giống IDA ở *chiến lược tái sinh node*. Nghiên cứu về RBFS mô tả IDA và RBFS là hai

paradigm quan trọng của linear-space heuristic search, đồng thời cho thấy regeneration overhead của RBFS phụ thuộc mạnh vào cấu trúc cây. ³

Lợi ích: rất phù hợp để dạy khái niệm `f-limit`, “best alternative” và đánh đổi memory/time. Độ khó: **trung bình**. Cần instrumentation riêng:

```
recursiveCalls
regeneratedNodes
maxRecursionDepth
fLimitUpdates
```

Đặc biệt nên có visualization cho biết tại sao RBFS quay lui và `f-limit` thay đổi thế nào.

SMA* — ưu tiên P1/P2. Đây là thuật toán đáng làm nếu muốn biến sản phẩm thành công cụ giảng dạy heuristic search chuyên sâu. Điểm khác biệt quan trọng là người dùng có thể đặt, chẳng hạn:

```
Memory budget = 50 MB
Memory budget = 100 MB
Memory budget = 500 MB
```

và quan sát cách hiệu năng thay đổi. Nghiên cứu SMA+ mô tả SMA như một hướng search có explicit memory bound và thảo luận cải tiến cấu trúc OPEN cũng như việc loại node khi bộ nhớ đạt giới hạn.

⁴

Độ khó: **cao**, vì implementation phải xử lý leaf eviction, lưu thông tin cần thiết để tái sinh subtree, cập nhật/backup giá trị `f` lên cha và đảm bảo semantics đúng khi memory bị giới hạn.

Weighted A*/bounded-suboptimal search — nên bổ sung trước ML. Với GUI thực tế, không phải người dùng lúc nào cũng cần lời giải tối ưu tuyệt đối. Có thể thêm:

$$f(n) = g(n) + w \times h(n), w \geq 1$$

để tạo “Fast solve mode”. Điều này cũng tạo cầu nối tự nhiên đến learned heuristic. Nghiên cứu bounded-suboptimal search cho thấy đây là một nhánh riêng rất phù hợp khi tối ưu thời gian/quality trade-off; các công trình gần đây cũng nghiên cứu cách khai thác learned heuristic trong focal/bounded-suboptimal search trên 15-puzzle. ²⁰

Nên báo cáo thêm:

```
optimalityGap = solutionCost / optimalCost
```

chứ không chỉ thời gian.

MCTS/UCT — P3. UCT của Kocsis và Szepesvári được đề xuất cho Monte-Carlo planning với state space lớn và xây dựng lựa chọn nhánh dựa trên bandit ideas. ⁸ Vì classical N-Puzzle deterministic, unit-cost và có heuristic lower bounds rất mạnh, tôi không khuyến nghị đầu tư MCTS trước IDA/PDB/RBFS. Đây là

suy luận từ cấu trúc bài toán và đặc tính của hai họ thuật toán*, không phải kết luận rằng MCTS “không thể” giải N-Puzzle.

MCTS chỉ trở nên hấp dẫn nếu bạn nghiên cứu:

- heuristic-free search so với heuristic-guided search;
- policy học bằng NN;
- action stochastic;
- puzzle generator/adversarial environments;
- “find some solution under fixed 100 ms” thay vì shortest solution.

Pattern Database — ưu tiên P1 rất cao. Thay vì chỉ cộng khoảng cách độc lập của từng tile, PDB giải chính xác một abstraction gồm nhiều tile rồi lưu kết quả vào bảng tra. Culberson–Schaeffer cho thấy đây là một cách tạo lower bound chặt hơn cho sliding-tile puzzles. ⁵

Pipeline nên là:

```
Goal state
  ↓
Chọn pattern {tile...}
  ↓
Reverse abstract search
  ↓
PDB[key(pattern positions)] = exact abstract cost
  ↓
Lưu file .pdb
  ↓
Load read-only khi solver khởi động
```

Mô-đun cần có:

```
PatternDefinition
PatternIndexer
PatternDatabaseBuilder
PatternDatabase
PatternDatabaseHeuristic
PdbMetadata
```

`PdbMetadata` phải lưu size puzzle, goal hash, tiles thuộc pattern, cost model, builder version và checksum. Như vậy không thể vô tình dùng PDB của goal state khác.

Disjoint/additive PDB — P1 cao nhưng sau single PDB. Korf–Felner chỉ ra rằng nếu bài toán con được phân hoạch sao cho chi phí operator được phân bổ phù hợp, các giá trị PDB có thể được cộng mà vẫn cho admissible heuristic. ⁶ Felner–Korf–Hanan còn phân biệt static partition và dynamic partition.

⁷

Một cấu hình thí nghiệm nên biểu diễn rõ:

```

heuristic:
  type: additive-pdb
  partition:
    - [1, 2, 3, 4, 5]
    - [6, 7, 8, 9, 10]
    - [11, 12, 13, 14, 15]

```

Không nên mặc định rằng “cộng nhiều heuristic càng tốt”. **Admissibility phải được chứng minh theo cost partition**, hoặc heuristic phải được gắn metadata `admissible=false`.

Linear Conflict mở rộng — cần ưu tiên correctness trước strength. Báo cáo hiện tại đã có H5 dựa trên Manhattan + linear conflict. Trong phiên bản nghiên cứu nên tránh cách “cứ thấy mỗi cặp conflict thì +2” nếu các conflict overlap mà chưa chứng minh không double-count extra moves. Thay vào đó, heuristic nên có test trên exact 3×3 và các instance nhỏ tự sinh để phát hiện vi phạm admissibility/consistency.

Đặc biệt, **H6 hiện tại nên được coi là heuristic thử nghiệm cho đến khi được chứng minh tự động/formal**. Báo cáo lập luận rằng cộng thêm “số ô không thể về đích” vào H5 vẫn admissible.

Tuy nhiên, từ góc nhìn lower-bound theory, việc `hA` và `hB` cùng admissible không tự động suy ra `hA+hB` admissible; phải chứng minh phần chi phí được cộng thêm không bị H5 tính trước. Literature về additive PDB phải đặt điều kiện partition chính vì vấn đề overcounting này. ⁷

Vì vậy heuristic API nên khai báo:

```

public interface Heuristic {
    String id();
    int estimate(Board state, Goal goal);

    default HeuristicProperties properties() {
        return HeuristicProperties.unknown();
    }
}

public record HeuristicProperties(
    boolean claimedAdmissible,
    boolean claimedConsistent,
    boolean requiresPreprocessing
) {}

```

Sau đó test suite xác minh claim đó.

Cách đánh giá heuristic chuyên nghiệp nên có hai lớp. Lớp intrinsic không chạy A*: so sánh trực tiếp với `h*`; lớp extrinsic chạy solver. Với 3×3, reverse BFS từ goal có thể tạo bảng exact distance cho mọi trạng thái reachable, từ đó tính:

Nhóm metric	Metric
Correctness	số lần <code>h > h*</code> , số consistency violations

Nhóm metric	Metric
Accuracy	<code>mean(h)</code> , <code>mean(h*/h)</code> khi thích hợp, <code>mean(h*-h)</code> , median error
Dominance	tỷ lệ state mà <code>hA > hB</code> , bằng nhau hoặc thấp hơn
Solver effect	expanded/generated/reopened/duplicate
Search quality	solution length/cost, optimality gap
Cost heuristic	ns/evaluation, tổng thời gian tính heuristic
Memory	PDB bytes, OPEN/CLOSED peak, heap peak
Aggregate	effective branching factor

Điểm quan trọng là **“heuristic mạnh hơn” không nhất thiết “solver chạy nhanh hơn”**: heuristic có thể giảm expansions nhưng mỗi evaluation quá đắt. PDB là ví dụ điển hình về trade-off giữa preprocessing/memory và số node search; chính literature PDB đánh giá hiệu quả thông qua giảm search effort. ²¹

Kiến trúc phần mềm

Kiến trúc nên thay đổi từ “ứng dụng JavaFX có thuật toán ở bên trong” thành **“search engine độc lập + JavaFX là một client”**. Đây là thay đổi có giá trị lớn nhất về maintainability.

Cấu trúc Maven đề xuất:

```

npuzzle-parent
├── npuzzle-core
│   ├── Board
│   ├── Goal
│   ├── Move
│   ├── PuzzleProblem
│   ├── Solvability
│   └── StateCodec
├── npuzzle-search-api
│   ├── SearchAlgorithm
│   ├── Heuristic
│   ├── SearchBudget
│   ├── SearchResult
│   ├── SearchMetrics
│   └── SearchObserver
└── npuzzle-algorithms
    ├── BFS
    ├── AStar
    ├── IDAStar
    ├── RBFS
    └── SMAStar

```

```

├── WeightedAStar
├── npuzzle-heuristics
│   ├── MisplacedTiles
│   ├── Manhattan
│   ├── LinearConflict
│   ├── PatternDatabase
│   └── AdditivePatternDatabase
├── npuzzle-benchmark
│   ├── Dataset
│   ├── ExperimentRunner
│   ├── CSVExporter
│   ├── BenchmarkManifest
│   └── jmh/
├── npuzzle-ui
│   ├── JavaFX controllers
│   ├── visualizer
│   ├── replay
│   └── compare
├── npuzzle-learning          // phase sau
└── npuzzle-distributed      // phase sau

```

Search API nên tối thiểu như sau:

```

public interface SearchAlgorithm {
    String id();

    SearchResult solve(
        PuzzleProblem problem,
        Heuristic heuristic,
        SearchBudget budget,
        SearchObserver observer
    );
}

public record SearchBudget(
    long timeoutMillis,
    long maxExpandedNodes,
    long maxMemoryBytes
) {}

public record SearchResult(
    SearchStatus status,
    List<Move> solution,
    SearchMetrics metrics
) {}

```

Lợi ích: GUI, CLI, benchmark và test đều gọi **cùng một solver implementation**, tránh tình trạng benchmark sử dụng code path khác với GUI.

Plugin thuật toán/heuristic. Java 17 `ServiceLoader` có thể tìm và nạp các service provider được đóng gói dưới dạng module hoặc JAR; service provider có thể được phát hiện mà application không cần phụ thuộc trực tiếp vào implementation cụ thể. ¹

Ví dụ:

```
ServiceLoader<SearchAlgorithm> algorithms =  
    ServiceLoader.load(SearchAlgorithm.class);  
  
for (SearchAlgorithm algorithm : algorithms) {  
    registry.register(algorithm);  
}
```

Một plugin bên ngoài chỉ cần:

```
npuzzle-plugin-my-search.jar  
META-INF/services/...SearchAlgorithm
```

hoặc khai báo `provides ... with ...` nếu dùng JPMS. Đây là hướng rất phù hợp để sinh viên có thể “viết thuật toán mới rồi thả JAR vào application”.

State representation hiệu năng cao. Với 3×3 và 4×4, mỗi tile có thể biểu diễn bằng 4 bit; 4×4 có đúng 16 ô nên toàn board có thể nằm trong 64 bit. Đây là một tối ưu toán học trực tiếp:

$$4 \text{ bit} \times 16 \text{ tiles} = 64 \text{ bit}$$

Do đó có thể dùng:

```
record PackedBoard4(long bits, byte blankIndex) {}
```

thay vì clone `int[]` cho mỗi successor.

5×5 có 25 giá trị 0–24, cần ít nhất 5 bit/ô:

$$5 \times 25 = 125 \text{ bit}$$

nên có thể dùng hai `long`, hoặc representation tổng quát hơn. Đừng ép tất cả size vào một encoding duy nhất. Thiết kế tốt hơn:

Board interface

```
|— PackedBoard64      // <= 4×4  
|— PackedBoard128     // 5×5
```

Lợi ích kỳ vọng là giảm object allocation, hash overhead và GC; **mức cải thiện thực tế phải được đo bằng JMH/JFR**, không nên tuyên bố trước. OpenJDK JMH khuyến nghị benchmark tách riêng và chạy trong môi trường kiểm soát. ⁹

Các tối ưu tiếp theo nên benchmark trước khi giữ lại:

```
precomputed blank moves  
goalRow[tile], goalCol[tile]  
incremental Manhattan update  
incremental linear-conflict update  
primitive/open-addressed closed set  
custom binary heap  
in-place DFS stack cho IDA*  
Zobrist/hash key cho distributed prototype
```

Không nên tối ưu mọi thứ cùng lúc. Chạy JMH cho bốn primitive operation:

```
encode/decode board  
generate successors  
hash>equals  
heuristic evaluation
```

sau đó mới quyết định custom data structure nào đáng giữ.

Đa luồng và async nên tách thành hai vấn đề khác nhau.

Vấn đề thứ nhất là **UI responsiveness**. JavaFX `Task` được thiết kế để chạy `call()` trên background thread và cung cấp observable state cho JavaFX; package `javafx.concurrent` tồn tại chính để chạy application work ngoài JavaFX event thread. ¹² Vì vậy mọi solver invocation từ controller nên đi qua `Task<SearchResult>` hoặc `Service<SearchResult>`.

Vấn đề thứ hai là **parallel computing**. Ở giai đoạn đầu, không nên parallelize một A đơn lẻ. *Hãy parallelize các experiment độc lập**:

```
Puzzle 1 → worker 1  
Puzzle 2 → worker 2  
Puzzle 3 → worker 3  
...
```

Java 17 `CompletableFuture` hỗ trợ asynchronous stages và cho phép truyền `Executor` riêng. Nếu không truyền executor, async methods thường dùng `ForkJoinPool.commonPool()`. ¹⁶ Với scientific benchmark, nên dùng executor riêng để số worker là một biến experimental rõ ràng:

```
ExecutorService executor =  
    Executors.newFixedThreadPool(threadCount);
```

thay vì vô tình chia sẻ common pool với các task khác.

Đến phase distributed search mới parallelize **một** search. Hash Distributed A* phân phối state/work theo hash giữa processing nodes; các nghiên cứu đã đánh giá cách tiếp cận này trên optimal planning và 24-puzzle ở quy mô nhiều core/machine. ²² Đây là hướng luận văn tốt nhưng không phải milestone đầu.

Observer/event architecture là cầu nối core với visualizer:

```
public interface SearchObserver {  
    void onExpand(SearchEvent e);  
    void onGenerate(SearchEvent e);  
    void onPrune(SearchEvent e);  
    void onGoal(SearchEvent e);  
}
```

Tuy nhiên không nên lưu mọi event của một search hàng triệu node. Cần ba mode:

```
OFF  
SAMPLED          // ví dụ 1 / K expansions  
FULL_TRACE       // chỉ dùng puzzle nhỏ
```

Như vậy benchmark mode không bị visualization làm sai lệch thời gian.

GUI/UX, benchmark và đo lường

GUI hiện tại đã có chơi puzzle, ảnh, lời giải và so sánh heuristic trong báo cáo gốc.

❌filecite❌turn0file0❌ Bước phát triển hợp lý không phải làm giao diện “đẹp hơn” trước, mà là chuyển nó thành **Search Laboratory**.

Visualizer bước-động. Màn hình nên có ba panel:

```
[ Puzzle / current state ]  
[ Frontier / current node metadata ]  
[ Metrics over time ]
```

Khi một node được expand, hiển thị:

```
g = 14  
h = 20  
f = 34  
depth = 14
```

```
expanded = 15,724
generated = 31,107
OPEN = 8,231
CLOSED = 15,724
```

Chế độ “Teaching” có thể tô khác các tile đang đóng góp Manhattan/linear conflict và giải thích vì sao **h** có giá trị hiện tại.

Replay cần tách thành hai loại.

Solution Replay chỉ phát lại đường đi lời giải:

```
Start → move → move → ... → Goal
```

Search Replay phát lại quá trình thuật toán expand/generate/prune. Loại thứ hai chỉ bật ở instance nhỏ hoặc sampled mode vì full trace có thể rất lớn.

Controls:

```
◀ step
▶ step
⏮ play/pause
slider speed
timeline
jump to expansion N
```

Heatmap có thể có ít nhất ba view:

- số lần blank xuất hiện ở từng ô;
- số lần một tile cụ thể xuất hiện ở vị trí;
- heatmap theo depth / **f** / **h** của các node expanded.

Đây là tính năng có giá trị giảng dạy cao vì nó làm rõ “heuristic kéo search về đâu”.

So sánh song song nên cho chọn:

```
A* + Manhattan
A* + Linear Conflict
IDA* + Manhattan
IDA* + PDB
```

tất cả dùng cùng **initialState**, **goal**, timeout và memory policy. Có thể đồng bộ replay theo số expansions hoặc wall-clock time.

JavaFX có sẵn họ **XYChart**, với các subclass như **LineChart**, **BarChart**, **ScatterChart** và **StackedBarChart**, đủ để xây phần lớn dashboard mà không cần chart framework ngoài. ¹⁵

Dashboard benchmark nên có:

```
BarChart    → nodes expanded
BarChart    → runtime
LineChart   → cumulative expansions vs time
ScatterChart → heuristic value vs true distance
LineChart   → memory vs time
```

Experiment Manager là tính năng tôi đánh giá P1 rất cao:

Dataset:

- ☐ easy-3x3
- ☐ exact-3x3
- ☐ random-15p
- ☐ hard-15p
- ☐ unsolvable

Algorithms:

- ☐ A*
- ☐ IDA*
- ☐ RBFS
- ☐ SMA*

Heuristics:

- ☐ Manhattan
- ☐ LinearConflict
- ☐ PDB
- ☐ AdditivePDB

Repetitions: 10

Timeout: 60 s

Heap: 2 GB

Seed: 20261006

Bấm **Run Experiment** và nhận một experiment folder hoàn chỉnh.

Format export đề xuất:

```
experiments/
  2027-01-22-ida-pdb/
    manifest.json
    raw-results.csv
    summary.csv
    puzzles.json
    environment.json
    charts/
    logs/
```

`manifest.json` nên có:

```
{
  "dataset": "15p-random-v1",
  "seed": 20261006,
  "algorithm": "IDAStar",
  "heuristic": "AdditivePDB",
  "timeoutSeconds": 60,
  "jdk": "17",
  "jvmArgs": ["-Xms2g", "-Xmx2g"],
  "repetitions": 10
}
```

Metrics bắt buộc cho mỗi run:

Metric	Ý nghĩa
<code>wallTimeNs</code>	Thời gian thực
<code>cpuTimeNs</code>	CPU time nếu đo được đáng tin
<code>expanded</code>	Node thực sự được expand
<code>generated</code>	Tổng successor sinh ra
<code>duplicates</code>	State bị nhận diện trùng
<code>reopened</code>	State được mở lại với <code>g</code> tốt hơn
<code>pruned</code>	Node bị loại
<code>maxOpen</code>	Peak frontier
<code>maxClosed</code>	Peak closed set
<code>solutionLength</code>	Số move
<code>solutionCost</code>	Cost, hiện tại thường bằng length
<code>heuristicCalls</code>	Số lần gọi <code>h</code>
<code>heuristicTimeNs</code>	Chi phí heuristic
<code>peakHeapBytes</code>	Peak heap
<code>gcCount</code> / <code>gcTime</code>	Tác động GC
<code>preprocessTime</code>	Ví dụ build/load PDB
<code>pdbBytes</code>	Chi phí storage heuristic

JDK Flight Recorder hỗ trợ thu và phân tích các event JVM; công cụ `jfr` có thể in hoặc tổng hợp recording và Oracle cũng hướng dẫn dùng Flight Recorder để phân tích các vấn đề GC/performance.

¹⁰ Không nên dùng JFR cho mọi unit test, nhưng rất hữu ích cho profile benchmark release.

JMH chỉ dùng cho microbenchmark, chẳng hạn:

```
@Benchmark
public int manhattan() { ... }

@Benchmark
public int linearConflict() { ... }

@Benchmark
public Board generateChild() { ... }

@Benchmark
public boolean closedSetLookup() { ... }
```

OpenJDK khuyến nghị Maven benchmark project độc lập và cảnh báo rằng JMH không nên được sử dụng như một thư viện test kiểu JUnit. ⁹ End-to-end solver benchmark vẫn phải dùng experiment runner riêng vì mục tiêu của nó là đo cả search process.

Reproducibility protocol nên bắt buộc lưu:

```
Git commit/tag
dataset checksum
PDB checksum
seed
OS
CPU model
RAM
JDK vendor/version
JVM arguments
Maven dependency versions
algorithm config
heuristic config
timeout/memory budget
warm-up policy
number of repetitions
raw results
```

Khi báo cáo time, nên giữ cả từng run thay vì chỉ lưu trung bình. Với dữ liệu search thường có outlier do scheduling/GC, nên report median và distribution bên cạnh mean; đây là khuyến nghị phương pháp luận để giảm rủi ro kết luận từ một lần chạy duy nhất.

CI integration. JUnit 5 hỗ trợ parameterized tests, nên có thể chạy cùng một invariant trên hàng trăm board/goal pair. ¹¹ Maven Surefire tích hợp test vào Maven lifecycle và sinh report; Failsafe có thể dành cho integration tests. ²³ JaCoCo có Maven plugin để tạo coverage report. ²⁴ GitHub cung cấp hướng dẫn chính thức cho CI Java/Maven và ví dụ `setup-java` với JDK 17 cùng `mvn ... verify`.

²⁵

Pipeline nên là:

```

Pull Request
↓
mvn verify
↓
unit tests
↓
solvability tests
↓
heuristic property tests
↓
optimality regression
↓
coverage
↓
small deterministic benchmark

```

Không nên để full 15-puzzle benchmark chạy trên mọi commit vì tốn thời gian và noisy. Full benchmark nên chạy nightly hoặc release.

Bộ thí nghiệm benchmark đề xuất

Thí nghiệm	Input cụ thể	So sánh	Metrics chính	Kỳ vọng / giả thuyết
Exhaustive heuristic verification	Reverse BFS từ goal 3×3, duyệt toàn bộ component reachable và lưu exact h^*	H1-H6, Manhattan, Linear Conflict, PDB	admissibility violations, consistency violations, h^*-h , dominance	H1/Manhattan chuẩn phải không overestimate; H5/H6 hiện có được kiểm chứng hoặc phát hiện counterexample
Optimal 15-Puzzle solver study	100 board 4×4 cố định; ưu tiên bộ benchmark công bố cùng nghiên cứu PDB hoặc một dataset fixed-seed được đóng gói trong repo	A, IDA, RBFS × Manhattan/LC/PDB	runtime, expanded, generated, peak memory, solution length	PDB giảm mạnh expansions; A <i>nhạy ở instance vừa nhưng chịu áp lực memory</i> ; IDA dùng ít RAM hơn. PDB đã cho hiệu quả rất lớn trong literature 15-Puzzle. ²¹
Hard-end stress test	Nhóm 4×4 instance có certified optimal depth cao; có thể thêm depth-80 cases khi dataset được xác minh	A, IDA, RBFS, SMA*	timeout rate, memory failure rate, expansions, regeneration	IDA/memory-bounded algorithms có lợi thế khi OPEN/CLOSED của A vượt budget. Đường kính 4×4 theo single-tile metric đã được nghiên cứu xác nhận là 80. ²⁶

Thí nghiệm	Input cụ thể	So sánh	Metrics chính	Kỳ vọng / giả thuyết
Solvability and arbitrary-goal test	Với mỗi size 3×3/4×4/5×5: 100 solvable boards + 100 boards tạo bằng cách swap hai numbered tiles; lặp lại với custom goals	Solvability checker + mọi solver	reject latency, expansions before rejection, false positive/negative	Unsolvable input phải bị loại trước search; goal tùy biến không được phá parity logic
Parallel experiment throughput	1.000 board 4×4 cố định; chạy 1, 2, 4, 8... worker tùy CPU	Sequential batch vs dedicated executor	puzzles/s, speedup, CPU utilization, GC, memory	Throughput tăng khi tăng worker đến khi CPU/memory/GC trở thành bottleneck; kết quả cho biết có nên đầu tư parallel single-search hay không

Thí nghiệm đầu tiên là quan trọng nhất về **khoa học**. Nó biến “chúng tôi tin heuristic này admissible” thành “chúng tôi đã kiểm tra exhaustively trên toàn bộ state component 3×3 và có regression test lưu trong repository”.

Thí nghiệm thứ hai là trung tâm của báo cáo/paper. Culberson–Schaeffer và Korf–Felner đã dùng sliding-tile puzzles để chứng minh mức tăng hiệu quả rất lớn của PDB và disjoint PDB, nên đây là baseline academic tự nhiên nhất. ²¹

Thí nghiệm thứ ba giúp tránh benchmark quá “dễ”. Công trình complete search gần đây xác nhận diameter của 4×4 sliding-tile puzzle là 80 single-tile moves, cho thấy có một vùng instance khó đủ mạnh để stress-test search architecture. ²⁷

Nghiên cứu, báo cáo và tính năng nâng cao

Để trở thành **sản phẩm nghiên cứu**, dự án cần chuyển từ “demo một số lần chạy” sang **hypothesis-driven experiment**.

Một chương kết quả tốt không nên chỉ có:

H1 < H2 < H5 . . .

mà nên trả lời những câu hỏi như:

Khi tăng độ chính xác của heuristic, số node expanded giảm bao nhiêu và chi phí evaluation tăng bao nhiêu?

Với cùng heuristic, IDA, RBFS và A đánh đổi thời gian/bộ nhớ ra sao?

Với memory budget cố định, SMA có vượt A bị memory cap hay không?

PDB kích thước lớn hơn đem lại lợi ích marginal thế nào?

Learned heuristic có giảm expansions đủ để bù chi phí inference và mất admissibility hay không?

Bảng kết quả chuẩn nên có dạng:

Solver	Heuristic	Solved	Median time	Median expanded	Peak memory	Mean length	Optimal
A*	Manhattan	...	...	...	...	...	100%
A*	Linear Conflict	...	...	...	...	...	100%
IDA*	Manhattan	...	...	...	...	...	100%
IDA*	PDB	...	...	...	...	...	100%
RBFS	PDB	...	...	...	...	...	...

Mỗi bảng cần ghi rõ timeout, heap, hardware, JDK và dataset version. Đây là phần giúp một người khác có thể tái tạo kết quả thay vì chỉ đọc kết luận.

Bộ biểu đồ nên chuẩn hóa gồm:

```
runtime vs instance
expanded nodes vs instance
peak memory vs instance
runtime vs optimal depth
expanded vs heuristic error
heuristic evaluation cost vs node reduction
PDB size vs runtime
memory budget vs SMA* runtime
threads vs benchmark throughput
```

Không nên chỉ dùng biểu đồ cột tổng hợp như báo cáo cũ. Báo cáo hiện tại đã có biểu đồ so sánh node và tỷ lệ tìm lời giải của H1–H6, đây là nền tốt, nhưng chưa đủ thông tin về dataset, repetitions, hardware, dispersion và raw results để trở thành benchmark tái tạo được. [filecite0turn0file0](#)

Các hướng bài báo/luận văn khả thi, xếp từ dễ đến khó:

Đề tài	Câu hỏi nghiên cứu	Độ khó
Time-Memory Trade-offs in Optimal N-Puzzle Search on the JVM	A/IDA/RBFS/SMA* khác nhau thế nào khi RAM bị giới hạn?	Trung bình
Evaluating Admissible Heuristics for Sliding-Tile Puzzles	Manhattan, LC, PDB, additive PDB khác nhau về accuracy/cost thế nào?	Trung bình

Đề tài	Câu hỏi nghiên cứu	Độ khó
Memory-Budget-Aware Pattern Database Selection	Với 64/128/256/... MB, partition PDB nào tốt nhất?	Cao
Explainable Heuristic Search Visualizer for AI Education	Visualization g/h/f, OPEN/CLOSED và replay giúp dạy heuristic search thế nào?	Trung bình
Learned Heuristics with Bounded-Suboptimal Search	NN heuristic có thể tăng tốc mà vẫn giữ solution-quality bound thế nào?	Cao
Parallel Hash-Distributed Search for Sliding-Tile Puzzles	HDA* scale thế nào trên multicore/ distributed JVM?	Rất cao

Learning heuristic bằng RL/NN nên được xây sau PDB. DeepCubeA cho thấy deep reinforcement learning kết hợp search có thể giải nhiều bài toán combinatorial puzzle, và công trình đó cũng đánh giá ngoài Rubik's Cube trên sliding-tile puzzle sizes. ²⁸ Nghiên cứu khác đã trực tiếp dùng learned heuristic cho 15-puzzle trong bounded-suboptimal focal search. ²⁹

Pipeline hợp lý:

```

Exact / solved states
    ↓
training examples
(state → distance-to-go / policy)
    ↓
Neural network
    ↓
LearnedHeuristic
    ↓
Weighted A* / Focal Search / IDA variant
    ↓
Compare with PDB

```

Không nên đưa learned h trực tiếp vào A* rồi gọi kết quả là optimal nếu chưa chứng minh admissibility. Thay vào đó báo cáo:

```

optimality gap
success rate under timeout
expansions
inference cost

```

Có thể nghiên cứu một hybrid:

```

admissible h = PDB
learned h    = ordering/tie-breaking signal

```

nhờ đó neural network hướng dẫn thứ tự search nhưng lower-bound logic vẫn dựa trên heuristic đã được xác minh. Đây là một hướng nghiên cứu an toàn hơn về correctness.

Hỗ trợ nhiều kích thước và custom goal nên được xem là một phần của core correctness, không phải chỉ UX. `Goal` phải là object:

```
public record Goal(Board board, int[] rowOf, int[] colOf) {}
```

Mọi Manhattan/PDB/solvability check phải nhận `Goal`, không được assume:

```
1 2 3
4 5 6
7 8 0
```

trong code.

Điều này sẽ tránh lặp lại dạng bug mà các heuristic phụ thuộc implicit vào thứ tự goal representation.

Adversarial puzzle generation là hướng nghiên cứu thú vị. “Adversarial” ở đây không nhất thiết là hai người chơi; có thể nghĩa là tự động tìm các state mà:

```
Manhattan thấp nhưng h* cao
Linear Conflict đánh giá kém
A* mở rất nhiều state
RBFS regeneration cực lớn
PDB partition A yếu nhưng partition B mạnh
```

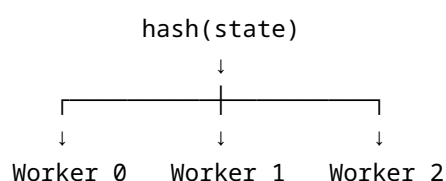
Có thể định nghĩa fitness:

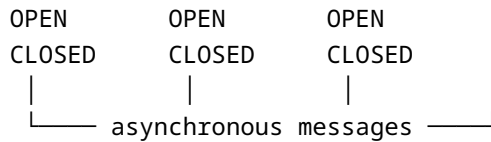
```
fitness(s) =
    expanded(A*, heuristic, s)
```

và dùng mutation/evolutionary search để tìm puzzle khiến heuristic gặp worst-case behavior. Đây là một đề tài tốt để minh họa rằng average benchmark chưa đủ.

Distributed search nên dùng HDA làm điểm xuất phát, thay vì tự phát minh protocol. HDA phân bổ state theo hash và đã được nghiên cứu trên distributed-memory systems cũng như 24-puzzle; kết quả cho thấy cách tiếp cận này có thể scale lên số lượng core lớn, dù hiệu quả phụ thuộc communication/load distribution. ³⁰ Công trình ICAPS trước đó cũng cho thấy hash-based work distribution có thể khai thác distributed RAM cho optimal best-first search. ³¹

Kiến trúc proof-of-concept:





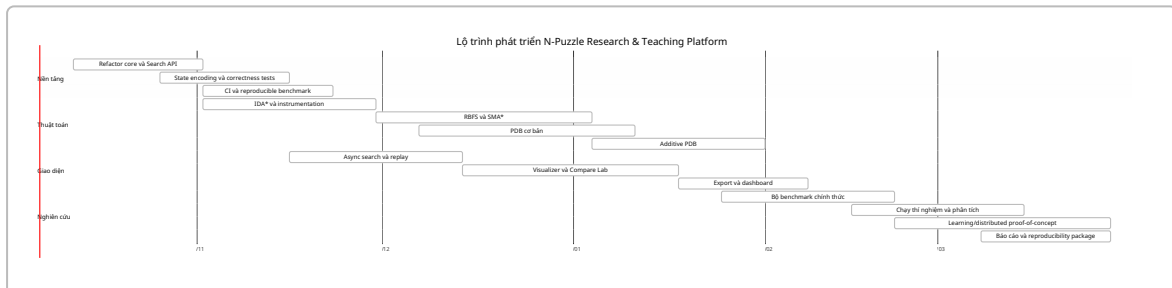
Metric riêng cần thêm:

messages sent
bytes transferred
states transferred
load imbalance
idle time
duplicate states across workers
parallel speedup

Đây mới là giai đoạn nên xem xét cluster/cloud. Trước đó, một PC multicore là đủ cho core/heuristic/PDB/benchmark work.

Kế hoạch triển khai

Lộ trình sau đây là **ước lượng kỹ thuật**, giả định một người phát triển khoảng 15–20 giờ mỗi tuần, codebase hiện tại đã chạy được Java 17 + Maven + JavaFX. Nếu làm đồ án theo nhóm hai người, có thể chạy song song UI và search engine, nhưng không nên giả định thời gian giảm đúng một nửa vì benchmark/correctness vẫn cần tích hợp chung.



Milestone nền tảng — khoảng ba đến năm tuần. Không thêm nhiều algorithm mới trong thời gian này. Mục tiêu là `npuzzle-core` không phụ thuộc JavaFX, mọi board immutable ở public API, goal được tổng quát hóa, solvability độc lập, search result chuẩn hóa và JUnit kiểm tra invariant. JUnit parameterized tests đặc biệt thích hợp để chạy cùng property trên tập input lớn. 11

Deliverable bắt buộc:

mvn verify → xanh
GUI vẫn chạy
CLI solver chạy
A* và BFS cho kết quả cũ
dataset/test fixtures versioned

Rủi ro lớn nhất là refactor đồng thời UI và algorithm gây regression. Giảm rủi ro bằng cách “đóng băng” một bộ golden puzzles với expected optimal lengths trước khi refactor.

Milestone optimal search — khoảng bốn đến sáu tuần. Hoàn thành IDA, RBFS và SearchMetrics. IDA phải được đối chiếu solution length với A trên toàn bộ test nhỏ. RBFS cần kiểm tra regenerated node statistics. Chỉ sau đó mới làm SMA.

Deliverable:

A*	optimal regression passed
IDA*	optimal regression passed
RBFS	optimal regression passed
SMA*	memory-budget regression passed

Rủi ro: lỗi subtle về duplicate/reopen/f-bound có thể vẫn sinh lời giải nhưng không còn optimal. Đây là lý do exact test oracle 3×3 phải được làm trước.

Milestone heuristic research — khoảng năm đến tám tuần. Đầu tiên standardize Manhattan và linear conflict, chạy exhaustive property test; sau đó single PDB; cuối cùng additive/disjoint PDB. Literature PDB cho thấy giá trị lớn nhưng cũng hàm ý trade-off preprocessing/storage đáng kể, nên size/partition phải là configuration chứ không hard-code. ²¹

Deliverable:

```
*.pdb format
PDB metadata/checksum
PDB builder CLI
PDB loader
PDB heuristic
AdditivePDB heuristic
admissibility test results
```

Rủi ro: file PDB lớn, indexing sai, goal mismatch, cost partition sai. Mỗi file PDB phải self-describing và có checksum.

Milestone teaching UI — khoảng năm đến sáu tuần, có thể chạy song song. Chuyển solver invocation sang Task / Service để search chạy ngoài UI thread; JavaFX cung cấp chính abstraction này cho background work và observable progress. ¹²

Deliverable:

```
Play
Solve
Pause/Cancel
Solution Replay
Search Replay
Step mode
```

```
g/h/f explanation
Compare side-by-side
Charts
```

Rủi ro: observer instrumentation làm chậm solver. Giải quyết bằng `OFF/SAMPLED/FULL_TRACE`.

Milestone benchmark — khoảng bốn tuần. Tạo `npuzzle-benchmark` riêng, JMH submodule riêng theo khuyến nghị của OpenJDK. ⁹ Thiết lập raw result schema và environment manifest; không để benchmark code phụ thuộc JavaFX.

Deliverable:

```
mvn verify
mvn -pl npuzzle-benchmark ...
benchmark result folder
raw-results.csv
manifest.json
charts
```

CI dùng Maven/Surefire cho correctness, GitHub Actions build JDK 17, JaCoCo cho coverage; full performance benchmark chỉ chạy scheduled/nightly hoặc thủ công vì CI host noise làm các con số timing khó so sánh tuyệt đối. GitHub có workflow chính thức cho Maven/JDK 17 và Maven verify. ²⁵

Milestone nghiên cứu nâng cao — khoảng bốn đến sáu tuần cho proof-of-concept, không phải production. Chọn **một** trong hai:

```
Learned Heuristic
hoặc
Parallel/Distributed Search
```

Không nên làm cả hai sâu trong cùng một đề án trừ khi có nhóm lớn.

Nếu chọn ML, baseline phải là PDB; DeepCubeA và nghiên cứu learned heuristics trên 15-puzzle cung cấp tiền lệ academic rõ ràng. ¹⁷

Nếu chọn distributed, baseline nên là HDA* hash-based ownership. ¹⁸

Tài nguyên nên chuẩn bị

Cho phần core/A/IDA/RBFS/UI, máy phát triển Java 17 thông thường là đủ. Cho PDB lớn, benchmark hard 15-puzzle và các run song song, nên có máy nhiều RAM; mức RAM cụ thể nên quyết định sau khi đã đo PDB footprint thay vì đặt một con số cố định từ đầu. Với learned heuristic, cần thêm Python/PyTorch hoặc một training stack tương đương; Java chỉ cần inference adapter nếu muốn giữ solver JVM-native. Với distributed search, cần tối thiểu hai process/worker chạy local trước khi chuyển sang nhiều máy.

Software stack đề xuất:

```

Java 17
Maven multi-module
OpenJFX
JUnit 5
Maven Surefire / Failsafe
JaCoCo
OpenJDK JMH
JDK Flight Recorder
GitHub Actions
CSV + JSON experiment artifacts

```

JUnit, Surefire, JaCoCo, JMH và GitHub Actions đều có tài liệu chính thức phù hợp trực tiếp với pipeline Java/Maven này. ³²

Rủi ro kỹ thuật quan trọng nhất

Rủi ro	Mức	Tác động	Biện pháp
Heuristic tưởng admissible nhưng thực tế overestimate	Rất cao	Kết quả “optimal” sai	Exhaustive 3×3 property test + proof
Double-count linear conflict/H6	Cao	A/IDA mất optimal guarantee	Tách experimental heuristic, verify <code>h≤h*</code>
PDB indexing/cost partition sai	Cao	Giá trị heuristic sai	Golden abstract states + metadata + checksum
Benchmark bị JIT/GC/noise làm sai lệch	Cao	Kết luận performance sai	JMH cho microbenchmark, repetitions, raw data, JFR ³³
UI observer làm chậm solver	Trung bình	Benchmark không còn đại diện	Observer OFF trong benchmark
A* hết RAM với hard 15-puzzle	Cao	Không hoàn thành dataset	IDA/RBFS/SMA + memory budgets
Parallel search làm thay đổi search order	Cao	Khó so với sequential	Batch parallel trước, HDA* sau
Learned heuristic phá admissibility	Cao	Mất optimality	Dùng bounded-suboptimal hoặc learned ordering ²⁹
Feature creep	Cao	Không hoàn thiện benchmark	P0/P1 freeze trước ML/MCTS/distributed

Mục tiêu sản phẩm cuối

Phiên bản “chuyên nghiệp” không nên được đánh giá bằng số lượng button hay số thuật toán, mà bằng khả năng trả lời một thí nghiệm từ đầu đến cuối:

Chọn dataset



```

Chọn solver
  ↓
Chọn heuristic
  ↓
Đặt timeout / memory / seed
  ↓
Run reproducibly
  ↓
Kiểm tra optimality/correctness
  ↓
Xuất raw metrics
  ↓
Tạo bảng và biểu đồ
  ↓
Replay một run cụ thể
  ↓
Người khác clone repo và tái tạo kết quả

```

Đó là điểm chuyển từ “**ứng dụng giải N-Puzzle**” sang “**N-Puzzle Heuristic Search Research & Teaching Platform**”.

Về mặt giá trị nghiên cứu, tổ hợp có tỷ lệ lợi ích/công sức cao nhất cho dự án của bạn là:

A + IDA + RBFS → Manhattan + Linear Conflict + PDB + Additive PDB → exact 3×3 verifier → reproducible 15-puzzle benchmark → Search Visualizer/Compare Lab.

Hướng này bám rất sát lịch sử và nền tảng học thuật của heuristic search: từ A* của Hart–Nilsson–Raphael, ¹⁹ qua iterative deepening của Korf, ² đến pattern databases của Culberson–Schaeffer và additive/disjoint PDB của Korf–Felner–Hanan. ³⁴ Đồng thời nó tận dụng tốt stack Java 17/Maven/JavaFX hiện tại thay vì buộc dự án phải đổi công nghệ.

ML, MCTS, GPU và distributed search nên được coi là **nhánh nghiên cứu sau khi nền benchmark đã đáng tin cậy**. Trong số đó, learned heuristic kết hợp bounded-suboptimal search có tiềm năng thành đề tài luận văn hiện đại nhất, còn HDA*/distributed search có chiều sâu hệ thống cao nhất. ³⁵ MCTS phù hợp nhất với vai trò algorithmic comparison/teaching baseline hơn là hướng tối ưu hóa solver N-Puzzle cổ điển, xét theo mục tiêu và miền bài toán ban đầu của UCT. ⁸

¹ ServiceLoader (Java SE 17 & JDK 17)

<https://docs.oracle.com/javase/jp/17/docs/api/java.base/java/util/ServiceLoader.html>

² Depth-first iterative-deepening: An optimal admissible tree search - ScienceDirect

<https://www.sciencedirect.com/science/article/pii/0004370285900840>

³ Recursive Best-First Search with Bounded Overhead | Proceedings of the AAAI Conference on Artificial Intelligence

<https://ojs.aaai.org/index.php/AAAI/article/view/9352>

⁴ Enhanced Simplified Memory-bounded A Star (SMA*+)

<https://easychair.org/publications/paper/TL2M>

- 5 21 34 **Pattern Databases - Culberson - 1998 - Computational Intelligence - Wiley Online Library**
<https://onlinelibrary.wiley.com/doi/abs/10.1111/0824-7935.00065>
- 6 14 **Disjoint pattern database heuristics - ScienceDirect**
<https://doi.org/10.1016/S0004-3702%2801%2900092-3>
- 7 **Additive Pattern Database Heuristics | ML Anthology**
<https://mlanthology.org/jair/2004/felner2004jair-additive/>
- 8 **Bandit based Monte-Carlo planning - Technion - Israel Institute of Technology**
<https://cris.technion.ac.il/en/publications/bandit-based-monte-carlo-planning/>
- 9 33 **GitHub - openjdk/jmh: <https://openjdk.org/projects/code-tools/jmh> · GitHub**
<https://github.com/openjdk/jmh>
- 10 **The jfr Command**
<https://docs.oracle.com/en/java/javase/17/docs/specs/man/jfr.html>
- 11 32 **JUnit 5 User Guide**
<https://junit.org/junit5/docs/current/user-guide/>
- 12 **Task (JavaFX 17)**
<https://openjfx.io/javadoc/17/javafx.graphics/javafx/concurrent/Task.html>
- 13 23 **Apache Maven Surefire – Surefire**
<https://maven.apache.org/surefire/index.html>
- 15 **XYChart (JavaFX 17)**
<https://openjfx.io/javadoc/17/javafx.controls/javafx/scene/chart/XYChart.html>
- 16 **CompletableFuture (Java SE 17 & JDK 17)**
<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/CompletableFuture.html>
- 17 28 **Solving the Rubik's cube with deep reinforcement learning and search | Nature Machine Intelligence**
<https://www.nature.com/articles/s42256-019-0070-z>
- 18 30 **On the Scaling Behavior of HDA* | Proceedings of the International Symposium on Combinatorial Search**
<https://ojs.aaai.org/index.php/SOCS/article/view/18163>
- 19 **A Formal Basis for the Heuristic Determination of Minimum Cost Paths | IEEE Journals & Magazine | IEEE Xplore**
<https://ieeexplore.ieee.org/abstract/document/4082128/authors>
- 20 **Bounded Suboptimal Search in Linear Space: New Results | Proceedings of the International Symposium on Combinatorial Search**
<https://ojs.aaai.org/index.php/SOCS/article/view/18327>
- 22 31 **Scalable, Parallel Best-First Search for Optimal Sequential Planning | Proceedings of the International Conference on Automated Planning and Scheduling**
<https://ojs.aaai.org/index.php/ICAPS/article/view/13350>
- 24 **JaCoCo - Maven Plug-in**
<https://www.jacoco.org/jacoco/trunk/doc/maven.html>
- 25 **Criar e testar o Java com o Maven - GitHub Docs**
<https://docs.github.com/pt/actions/use-cases-and-examples/building-and-testing/building-and-testing-java-with-maven>

26 27 Complete Search of Sliding Tile Puzzles on a Personal Computer [Extended Abstract] |
Proceedings of the International Symposium on Combinatorial Search
<https://ojs.aaai.org/index.php/SOCS/article/view/27307>

29 35 Focal Discrepancy Search for Learned Heuristics (Extended Abstract) | Proceedings of the
International Symposium on Combinatorial Search
<https://ojs.aaai.org/index.php/SOCS/article/view/21786>